

Project 3 Q&A; Reference Guide

Retail Sales Forecasting Pipeline

Project 3: Retail Sales Forecasting

Q1: Explain the overall execution pipeline and architecture of the Retail Sales Forecasting project.

Hinglish Explanation (WhatsApp Language):

Is project mein humne modular phases banaye hain:

1. **data_loader.py** daily records aggregate karke CSV banata hai.
2. **feature_engineering.py** calendar variables, lags aur rolling mean add karta hai.
3. **train_ml_models.py** sorted timeline split apply karke Random Forest aur XGBoost models train karta hai aur joblib files save karta hai.
4. **evaluate_models.py** calculations (MAE, RMSE, R-squared) test set par run karke metrics compare karta hai.
5. Streamlit app dashboards forecast dynamically plots visualize karti hai.

Formal Answer (English):

The project implements a modular software engineering workflow:

1. **Data Aggregation (**data_loader.py**)**: Loads transaction records and aggregates them to a daily level.
2. **Feature Engineering (**feature_engineering.py**)**: Computes chronological lags, rolling means, rolling standard deviations, calendar attributes, and differences.
3. **Training (**train_ml_models.py**)**: Splitting chronologically (80% train, 20% test). Trains Random Forest and XGBoost regressors, saving models using **joblib**.
4. **Evaluation (**evaluate_models.py**)**: Generates comparative metrics (MAE, RMSE, R^2) to identify the best model.
5. **Dashboard (**app/streamlit_app.py**)**: Visualizes forecasting performance and lets users compare historical trends with predictive futures.

New Tech / Key Concept:

- **Modular Pipeline**: Decoupling data ingestion, feature generation, training, and UI into isolated scripts to simplify maintenance and testing.

Q2: Detail the feature engineering strategy used in **feature_engineering.py and how it prevents data leakage.**

Hinglish Explanation (WhatsApp Language):

Time-series forecasting models future direct prices target nahi kar sakte, unhe past details ki zarurat hoti hai. Humne features design kiya hai:

* **Calendar indicators:** month, day_of_week aur weekend check parameters.

* **Shift Lags:** shift metrics lag_1, lag_7, lag_14, lag_30.

* **Rolling Stats:** 7-day average trends and 30-day volatility standard deviations.

*Leakage rokne ke liye rolling aur lag calculation strictly **.shift()** use karte hain takki future data past points mein mix na ho.*

Formal Answer (English):

Tabular forecasting models require engineered temporal indicators:

- **Calendar Features:** Extracts **month**, **day_of_week**, and **is_weekend** to represent recurring shopping cycles.
- **Lags:** Captures immediate historical momentum (shifted by 1, 7, 14, and 30 days).
- **Rolling Statistics:** Computes rolling means and standard deviations across 7 and 30-day windows to represent trend and volatility.
- **Preventing Data Leakage:** All rolling calculations are preceded by **.shift(1)** to ensure the feature for day t is calculated strictly using indices $t-1$ and older, preventing lookahead bias.

New Tech / Key Concept:

- **Lag Features:** Shifting historical variables backward in time so the model can evaluate what happened in previous intervals to predict the future.

Q3: Why is standard K-Fold cross-validation avoided in this project? What strategy is used instead?

Hinglish Explanation (WhatsApp Language):

Standard K-Fold CV mein data randomly groups mein shuffle ho jata hai. Time-series data mein agar sequence shuffle ho jaye, toh 'future' data training points mein aa jata hai aur model 'past' ko target kar leta hai, jisse leakage hoti hai aur testing evaluation results over-inflated (fake high accuracy) dikhte hain. Isliye humne bina shuffle kiye chronological split kiya hai, jahan starting 80% data training aur end 20% validation ke liye use hota hai.

Formal Answer (English):

Standard K-Fold Cross-Validation shuffles data randomly. In time-series forecasting, this breaks the sequential ordering of transactions:

1. **Temporal Dependency Violation:** Shuffling removes the chronological dependency of the records.

2. **Lookahead Data Leakage:** Future records are grouped into training folds, allowing the model to utilize future information to predict past targets.
- **Strategy Used:** We implement a chronological split. The dataset is sorted by date, and the split is performed sequentially (first 80% for training, final 20% for testing), simulating how the model will process new incoming transaction logs in production.

New Tech / Key Concept:

- **Lookahead Data Leakage:** An evaluation error where testing/future parameters leak into the training process, leading to unrealistic validation metrics.

Q4: Compare the performance of the Random Forest and XGBoost models. How do you interpret the R^2 score?

Hinglish Explanation (WhatsApp Language):

RF aur XGBoost dono models tree ensemble use karte hain but different ways mein:

* **Random Forest:** Bagging technique use karta hai, jahan parallel trees independent subsets par fit ho kar average prediction deti hain (overfitting control).

* **XGBoost:** Boosting use karta hai jo tree structures sequential banata hai, matlab har naya tree pichle predictions ke errors ko seekhta aur thik karta hai.

XGBoost ne overall score raising accuracy raise ki ($R^2 = 0.933$). Iska matlab hai ki 93.3% retail revenue variations humare features explain kar sakte hain, aur baaki 6.7% random noise hai.

Formal Answer (English):

- **Random Forest Regressor:** Uses **Bagging** (Bootstrap Aggregating). Builds multiple independent trees in parallel and averages their outputs, which reduces variance and makes the model robust.
- **XGBoost Regressor:** Uses **Boosting**. Builds weak decision trees sequentially, with each new tree minimizing the gradient loss (residuals) of the preceding models. XGBoost performed better, yielding lower MAE and RMSE, and achieving an R^2 score of 0.933.
- **R^2 Interpretation (0.933):** Means 93.3% of the variance in daily retail revenue is explained by our input features (lags, rolling averages, calendar elements). Only 6.7% remains unexplained, which represents unmeasured random noise.

New Tech / Key Concept:

- **R^2 (Coefficient of Determination):** A statistical metric that represents the proportion of variance in the dependent variable that is predictable from the independent variables.

Q5: What are the specific hyperparameter configurations used for Random Forest and XGBoost in `train_ml_models.py`?

Hinglish Explanation (WhatsApp Language):

Humne dono models ke overfitting check limits handle karne ke liye configurations optimize kiye hain:

** Random Forest mein depth boundary 12 aur estimators 200 select kiya hai.*

** XGBoost mein depth limits 6 aur learning rate 0.05 targets dynamic scaling parameters. Isse model learning stable rehti hai.*

Formal Answer (English):

The hyperparameters are configured in **train_ml_models.py** to balance model capacity and prevent overfitting:

- **Random Forest Regressor:**
 - **n_estimators=200**: Number of decision trees.
 - **max_depth=12**: Limits tree growth to prevent individual trees from memorizing noise.
 - **min_samples_split=5**: Minimum samples required to split an internal node.
- **XGBoost Regressor (XGBRegressor):**
 - **n_estimators=300**: Sequential boosting steps.
 - **max_depth=6**: Shallow depth limits compared to RF because boosting builds sequentially.
 - **learning_rate=0.05**: Step size shrinkage.
 - **subsample=0.8** & **colsample_bytree=0.8**: Ratios of row and feature sampling to introduce randomness.

New Tech / Key Concept:

- **Subsample (XGBoost)**: The ratio of training instances randomly sampled at each step, introducing stochastic gradient descent benefits.

Q6: Detail the metrics evaluated in **evaluate_models.py and explain why MAE and RMSE are evaluated together.**

Hinglish Explanation (WhatsApp Language):

MAE predictions aur actual inputs ka absolute distance measure karta hai (simple average error). RMSE error values ka square karke mean nikalta hai aur fir root karta hai. Agar dono metrics compare karein, aur RMSE value MAE se bahot badi hai, iska matlab project mein occasional massive errors (demand spikes) ho rahi hain jo model control nahi kar paa raha.

Formal Answer (English):

- **MAE (Mean Absolute Error)**: Measures the average magnitude of prediction errors. Every error is weighted linearly.
- **RMSE (Root Mean Squared Error)**: Computes the square root of the mean of squared differences. Because errors are squared before averaging, **RMSE penalizes larger errors more heavily**.
- **Evaluating Together**: Comparing them tells us about error variance. If RMSE (\$240.12) is significantly higher than MAE (\$153.10), it indicates the presence of large outlier forecasting errors on specific days

(e.g., promotional campaigns or stockouts) that we may need to address with targeted features.

New Tech / Key Concept:

- **RMSE vs. MAE:** The relative gap between these metrics indicates the frequency and severity of large prediction outliers.

Q7: Explain the recursive multi-step forecasting strategy. How do lag features propagate over a 30-day horizon?

Hinglish Explanation (WhatsApp Language):

Time-series models lagged variables (jaise kal ki sales) use karte hain. Jab hume 30 days future forecast karna ho, toh Day 2 predict karte waqt Day 1 ki actual sales available nahi hogi. Hum recursive approach use karte hain: Day 1 ka prediction compute karke use history mein append karte hain, lags re-calculate karte hain, aur fir Day 2 predict karte hain. Yeh recursion Day 30 tak repeat hota hai.

Formal Answer (English):

Because our models depend on lag features (like **lag_1** and **lag_7**), we cannot predict multiple steps into the future in one pass, as actual sales values for future steps are unavailable.

- **Mechanism:**

1. Predict sales for Day 1 (\$t+1\$).
2. Append the predicted value to the historical dataset.
3. Re-calculate all lag, rolling, and momentum features using this prediction.
4. Predict sales for Day 2 (\$t+2\$).
5. Repeat recursively until the entire 30-day forecast horizon is built.

- **Risk:** Prediction errors accumulate over the horizon, making predictions further in the future progressively less accurate.

New Tech / Key Concept:

- **Recursive Multi-Step Forecasting:** Sequentially generating forecasts where predicted values are recycled as features for future time steps.

Q8: How does the Streamlit forecasting application load models and visualize results?

Hinglish Explanation (WhatsApp Language):

*Python ML scripts models ko **.pkl** files mein serialise karti hain. Streamlit dashboard initialize hote hi **joblib.load()** use karke models load kar leta hai. User user-input parameters select karta hai, predictions generate hoti hain, aur Seaborn/Matplotlib plots banakar daily retail trends visually compare karte hain dynamically.*

Formal Answer (English):

- **Model Loading:** The Streamlit application deserializes the trained Random Forest and XGBoost models from `models/` using `joblib.load()`.
- **Dynamic Prediction:** When a user selects a forecast date range, the application slices the recent historical records, processes them through the feature pipeline, and executes model inferences.
- **Visualization:** Renders comparative line charts overlaying historical sales and forecasted revenue, highlighting performance metrics (MAE, RMSE, R^2) to help stakeholders evaluate predictions.

New Tech / Key Concept:

- **Joblib:** A Python library optimized for caching and serializing large NumPy arrays and Scikit-Learn models quickly.